Admissibility Engineering

Governing Agentic Software Realization Through Constraint Binding and Specification Saturation

A realization should not become acceptable merely because an executor found a path the specification failed to constrain.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Agentic software construction can produce implementations that compile, run, and pass the available tests while violating an invariant that the specification intended to preserve. This paper calls that family **plausible-but-wrong realization** and introduces **admissibility engineering** as the discipline of reducing it by binding important constraints into independently checkable specification, configuration, implementation, and evidence artifacts.

The paper formalizes saturation relative to a declared architecture and boundary. For invariant I in architecture A, the realization surface $R_A(I)$ contains the in-scope pathways through which I may be satisfied or violated; the evidenced constraint surface $C_A(S, I, E)$ contains the pathways whose constraint is supported by specification S and evidence E. Their difference is the **observed saturation gap**. Because undeclared or undiscovered pathways may remain, an empty observed gap is not proof of absolute completeness. A saturation claim must name its architecture, capability boundary, threat model, enumeration method, evidence, tolerable ambiguity, and residual unknowns.

Artifact-layer techniques–interfaces, type and schema constraints, executable contracts, reference models, behavioral oracles, mutation tests, capability mediation, and trace obligations–can reduce interpretive freedom at selected boundaries. None is self-validating: an artifact may encode the wrong invariant, cover only shape, or be bypassed. Admissibility engineering therefore composes constraints with independent observation and counterexample search rather than replacing tests with artifacts.

Receipts are treated as governed evidence objects, not as behavior itself. They can bind declared transitions, inputs, decisions, and observations, but they establish an effect only when the receiving claim is supported by appropriate mediation coverage, witness, recomputation, or independent state observation.

The resulting evaluation model distinguishes exact-artifact equality, trace equivalence, behavioral equivalence, and obligation satisfaction instead of collapsing them into one convergence claim.

The present paper supplies a theory, design discipline, and empirical protocol. It does not report a completed experiment because the exact current measurements and bound evidence package are not supplied in this source head. Dominance, threshold, comparative-performance, and cross-executor claims therefore remain hypotheses to be tested rather than results to be backfilled from memory.

---

1. The Failure Mode

1.1 Three Failure Classes

Agentic coding systems–autonomous LLM-driven software construction loops that read repositories, modify files, execute tests, and iteratively refine–fail in qualitatively distinct ways. Conflating these failure modes obscures what kind of intervention is required to address each.

The three failure classes are:

Hallucination. The model generates references to APIs, libraries, types, or behaviors that do not exist. This failure is loud: the resulting code typically does not compile or fails immediately at runtime. Conventional static analysis catches most hallucinations; the rest surface in the first execution.

Syntax failure. The generated code is structurally invalid–malformed parses, type errors, missing imports, mismatched brackets. Like hallucination, this failure is loud: compilers and interpreters reject the output before behavior is observable.

Plausible-but-wrong realization. The generated code is structurally valid, type-correct, syntactically well-formed, and superficially aligned with the specification. It compiles, runs, and frequently passes the tests written against it. Yet its behavior diverges from the specification's intent in ways that are visible only under behavioral truth examination.

The first two failure modes are often exposed by compilers, type checkers, dependency resolution, and execution, but they are not solved in general. Plausible-but-wrong realization can remain latent because the implementation agrees with the checks that exist while violating an unstated, mis-stated, or uncovered obligation. Whether it is the dominant failure class is an empirical question for a declared task distribution and measurement protocol.

1.2 Why Plausible-But-Wrong Can Persist

As task and system complexity increase, the realization space available to an agentic executor can expand beyond what a short human-readable specification

explicitly constrains. A specification stating "the function shall return the sorted list" admits many realizations:

- a correct sort implementation
- a sort that mutates input rather than returning a fresh list
- a sort that returns a fresh list but discards duplicate elements silently
- a sort that depends on incidental ordering properties of the input
- a sort whose comparator is unstable
- a sort that succeeds for the test inputs but fails for edge cases the test suite did not exercise

An underspecified phrase may fail to rule out several of these variants. Whether any variant satisfies the specification depends on requirements the phrase leaves unstated–mutation policy, duplicate preservation, comparator semantics, stability, and domain boundaries. The example demonstrates specification ambiguity; it does not by itself prove a single recoverable authorial intent.

This failure mode can be invisible to a unit-test suite whose cases and oracles do not exercise the violated obligation. A plausible-but-wrong implementation survives when it satisfies the tests as written while violating a requirement those tests and other checks do not encode.

1.3 Concrete Failure Examples

Concrete failure classes recurring across realization paths of substantial governed systems include:

Sort stability bug class. An implementation passes ordering tests because the input data happens to be presorted on a secondary key. The sort is unstable; the test never exposes this because the inputs are stable on that key by coincidence.

Receipt-wiring gap. A receipt-bound capability is declared and the action executes successfully; tests confirm the action produced the correct primary effect. But the receipt is not actually wired into the lineage graph. The system passes its tests; its lineage record is silently broken.

Authority laundering bug. A function checks authority at entry and passes the validated authority object to a helper; the helper does not re-validate but uses the authority object directly. Refactoring later moves the helper to a different invocation site where the entry-check no longer applies. Authority is laundered through the helper. No test exercises this because at the time of test authoring, the entry-check covered the helper's only call site.

Plausible API names. The model generates calls to API surfaces that *should* exist by naming convention but do not exist in the actual library. Often caught by compilers, but not always–particularly in dynamically typed languages or when method dispatch uses string-based reflection.

These are source-reported failure examples that motivate the taxonomy. Their frequency, representativeness, and dominance require the bound empirical record

specified in Section 5.

1.4 Tests Are Necessary but Not Exhaustive

Behavioral tests are indispensable evidence. Example tests, property-based generation, metamorphic relations, fuzzing, mutation testing, model checking, differential testing, and runtime monitoring can each expose different parts of a realization surface. No finite suite establishes unrestricted semantic correctness merely by passing.

Admissibility engineering adds independent constraint surfaces; it does not replace testing. A type can enforce a shape while permitting the wrong behavior. An abstract base class can require a method while saying nothing about its semantics. A contract can encode the wrong postcondition. A receipt can be complete relative to an instrumented pathway while an unmediated pathway bypasses it.

The desired structure is therefore plural:

1. declare the invariant and its scope;
2. enumerate known realization pathways;
3. bind the invariant at the strongest useful layer;
4. test the binding with positive, negative, adversarial, and mutation cases;
5. observe effects independently where consequence requires it;
6. record residual unknowns and revise the pathway inventory when counterexamples appear.

The target is not a system that needs fewer kinds of evidence. It is a system in which no single check is allowed to impersonate complete assurance.

---

2. Specification Saturation

2.1 Architecture-Relative Realization Surface

Let A be a declared architecture and I an invariant. The **realization surface**

$$R_A(I)$$

is the set of enumerated in-scope pathways in A through which I may be satisfied or violated. A pathway may be a code path, state transition, capability edge, serialization boundary, configuration choice, retry route, error path, migration, or external effect surface. The subscript is essential: refactoring, configuration change, provider substitution, or newly discovered bypass can change the surface without changing the prose statement of I.

The surface is an evidence-bearing inventory, not an omniscient set. A review must state how pathways were discovered and which capability surfaces were outside observation.

2.2 Evidenced Constraint Surface

Let S be the specification and configuration artifacts intended to preserve I, and let E be the available evidence. Define

$$C_A(S, I, E) \subseteq R_A(I)$$

as the pathways for which the claimed constraint is both applicable and supported by the declared evidence. A prose obligation, type, interface, contract, test, mediator, or receipt contributes only the coverage it can demonstrate. Artifact existence alone does not place a pathway in C_A.

2.3 Observed Saturation Gap and Unknown Surface

The **observed saturation gap** is

$$G_A(S, I, E) = R_A(I) \setminus C_A(S, I, E).$$

Plausible-but-wrong realizations may inhabit this gap. In addition, let $U_A(I)$ denote pathways that exist but are absent from the current inventory. Since $U_A(I)$ is only discovered through new evidence, an observed empty gap does not prove absolute saturation:

$$G_A = \varnothing \not\Rightarrow U_A = \varnothing.$$

Any completeness statement is therefore relative to an architecture revision, capability boundary, enumeration procedure, threat model, evidence package, and time.

2.4 Complexity Threshold as a Hypothesis

As component count, interaction depth, concurrency, configurability, and interface multiplicity increase, the cost of enumerating and constraining realization pathways may rise nonlinearly. Prose-only governance may then become too ambiguous or expensive to maintain. The paper calls the point at which additional artifact-bound constraints become cost-effective a **complexity threshold**.

This is not asserted as a universal constant or phase law. It is a deployment-relative hypothesis. Evidence requires a declared complexity measure, observed defect classes, specification forms, comparable tasks, and intervention results. The threshold may differ across languages, teams, domains, tools, and consequence classes.

2.5 Tolerable Ambiguity

Not every unconstrained choice is a defect. Algorithm selection, naming, internal decomposition, optimization, or presentation may remain open when all governing invariants are preserved. These are **tolerable ambiguity regions**.

Tolerable ambiguity must be declared with its scope and non-negotiable boundary. Otherwise a later reviewer cannot distinguish intended executor freedom from an undiscovered gap. Ambiguity also remains revisable: operational evidence may reveal that a previously harmless degree of freedom affects security, interoperability, reproducibility, or cost.

2.6 Operational Saturation

Absolute ideal saturation is generally unavailable as an empirical claim. **Operational saturation** is the stronger usable target: for a declared architecture revision and consequence class,

- the in-scope pathway inventory is current under the selected discovery method;
- every enumerated pathway has a justified constraint or declared tolerable ambiguity;
- negative and mutation tests challenge each binding;
- bypass and capability-surface discovery seek unenumerated pathways;
- residual unknowns and evidence limitations are explicit;
- new counterexamples reopen the claim.

Operational saturation is thus a defeasible evidence state, not a permanent property conferred by the specification's author.

---

3. Admissibility Engineering

3.1 Definition

Admissibility engineering is the discipline of identifying consequence-bearing realization pathways, binding their invariants into independently inspectable artifacts and runtime boundaries, and testing whether inadmissible realizations are refused before effect or explicitly classified as unverified.

The discipline spans specification, configuration, implementation, evidence, and revision. It does not mean that every constraint belongs in code or that mechanical enforcement is always preferable. The appropriate layer depends on what must be constrained, who may revise it, what evidence exists, and which failures are tolerable.

3.2 Constraint Binding

Constraint binding connects a named invariant to a defined enforcement or evaluation point. A useful binding identifies:

- the exact invariant and consequence class;
- the architecture and configuration revision;
- the pathways it covers;
- the mechanism that admits, refuses, or observes them;
- authority to define and revise the mechanism;
- expected evidence and failure codes;
- known bypasses and residual unknowns.

A binding may act at authoring, compilation, construction, deployment, invocation, transition, effect, or post-effect verification time. Calling every artifact-layer check "construction-time" would obscure these distinct assurance moments.

3.3 Artifact and Evidence Techniques

- **Technique:** Abstract interfaces and projected ABCs
  Useful constraint: Required operation shape and dependency direction
  Important limit: Presence of a method does not establish semantic behavior.
- **Technique:** Type and schema scaffolds
  Useful constraint: Representable values, field obligations, and some state distinctions
  Important limit: Many semantic and temporal obligations remain expressible but unenforced.
- **Technique:** Preconditions, postconditions, and invariants
  Useful constraint: Local transition obligations
  Important limit: A contract may be wrong, disabled, or bypassed.
- **Technique:** Typestate and capability APIs
  Useful constraint: Permitted operation order and authority attenuation
  Important limit: Runtime bindings and hidden side channels still require evidence.
- **Technique:** Reference models and behavioral oracles
  Useful constraint: Expected externally observable behavior
  Important limit: Oracle independence and completeness must be justified.
- **Technique:** Property, metamorphic, fuzz, and mutation tests
  Useful constraint: Broad counterexample search and test sensitivity
  Important limit: Sampling does not prove exhaustion.
- **Technique:** Structural and dependency checks
  Useful constraint: Cross-component topology and forbidden adjacency

Important limit: Static structure may diverge from dynamic execution.

- **Technique:** Receipt and trace obligations
  Useful constraint: Declared transition, lineage, and observation evidence
  Important limit: Self-reports do not prove mediation coverage or external effect.
- **Technique:** Named adjacency targets
  Useful constraint: A concrete nearby wrong system the binding must exclude
  Important limit: The named list may omit novel failures.

The catalog is intentionally heterogeneous. Saturation improves when independent techniques overlap in consequence-bearing regions and disagree detectably when one layer drifts.

3.4 Constraint Relocation and Its Risks

Moving a rule from prose into an artifact can make a boundary more precise and testable, but relocation is not automatically an improvement. It may freeze a mistaken interpretation, hide policy in code, narrow legitimate implementations, increase migration cost, or transfer authority from an author to a toolchain. Every relocation therefore requires provenance, authority, scope, revision, and rollback semantics.

The goal is not maximum constraint. It is the least restrictive set of bindings that excludes identified harmful or incorrect realization classes while preserving declared legitimate variation.

3.5 Relationship to Constraint-Bound Inference

Constraint-Bound Inference (CBI) describes an executor operating within declared bounds. Admissibility engineering constructs, tests, and revises those bounds. CBI is an execution posture; AE is the engineering discipline that asks whether the posture's constraint surface is adequate for a particular architecture and consequence class.

Neither term grants conformance. An executor can claim to be constrained while important pathways remain unbound, and a rich artifact package can exist while the runtime ignores it.

---

4. Receipts as a Governed Evidence Substrate

4.1 Evidence, Not Behavior

Receipts can make system claims durable, attributable, and recomputable. They can bind an input identity, admitted configuration, transition request, decision,

expected effect, observation, and predecessor relation. This makes a receipt chain a valuable **governed evidence substrate**.

A receipt is not the behavior it describes. A system may emit a success receipt before an effect, fabricate an observation, omit an unmediated pathway, or faithfully record an incorrect decision. The relevant chain is therefore:

$$\begin{aligned}&\text{Obligation}\\ &\rightarrow \text{AdmittedConfiguration}\\ &\rightarrow \text{Transition}\\ &\rightarrow \text{EffectObservation}\\ &\rightarrow \text{Receipt}\\ &\rightarrow \text{IndependentVerification}\end{aligned}.$$

Each arrow needs its own evidence and failure semantics. Cryptographic integrity can protect a receipt after creation; it cannot make the underlying claim true or complete.

4.2 Receipt-Backed Test Oracles

Tests may use receipts as inputs to an oracle when the receipt schema exposes the transition evidence under examination. Stronger tests combine:

- receipt validation and predecessor continuity;
- independent observation of relevant state or effect;
- recomputation from exact inputs where deterministic;
- deliberate bypass and omission attacks;
- mutation of receipts and runtime bindings;
- negative cases that must produce typed refusal.

This avoids both extremes: inspecting only mutable in-memory state, and trusting a self-consistent receipt chain as proof of reality.

4.3 Mediation and Completeness

A receipt-backed completeness claim requires evidence that every in-scope effect pathway crosses the receipt-producing boundary. That may require capability mediation, operating-system or hardware observation, external witnesses, reconciliation against independent logs, or destructive bypass tests. The strength of the claim cannot exceed the strength of that coverage evidence.

4.4 Verification Harness Standing

The source describes a `HeadlessRunner` and

`ScenarioResult` pattern. In this candidate they remain an implementation profile to be bound to an exact artifact and evidence package. Their names do

not establish that the harness observes real effects, resists stubbing, or covers every relevant pathway. A publication-ready empirical section must identify the exact harness revision, schemas, runner environment, fault injections, expected failures, and independently verified outputs.

4.5 Corpus Integration

Receipt-Bound Capabilities binds permission to evidence obligations. *Reasoning by Way of Receipts* extends the model to intermediate epistemic transitions. *The Three Hash Rule* separates state, admissibility, and meaning commitments. Semantic Custody preserves who holds and may transform semantic material. Admissibility engineering contributes the complementary pathway question: which realized routes must be constrained or observed before any of those commitments can support a bounded claim?

---

5. Evidence Contract and Current Status

5.1 Current Standing

This source head contains no publication-grade empirical package. Earlier scale counts and convergence examples were explicitly marked as stale placeholders and are not carried forward as results. The conceptual framework can be evaluated now; empirical claims remain pending exact evidence.

5.2 Required Experiment Record

Each reported build should bind:

- codex or specification identity and exact revision;
- Instance Configuration and implementation target;
- source and dependency commitments;
- executor, model, toolchain, environment, and authority surface;
- attempt, retry, intervention, and specification-amendment history;
- invariant catalog and realization-path inventory;
- constraint-to-path coverage matrix;
- test, mutation, fault-injection, and bypass corpus identities;
- effect observations and receipt commitments;
- failures, exclusions, timeouts, and missing evidence;
- final comparison relation and its decision procedure.

5.3 Saturation Metrics

For enumerated in-scope pathways,

$$\text{ObservedCoverage}(I) = \frac{|C_A(S, I, E)|}{|R_A(I)|}.$$

The denominator must state the enumeration method and revision. A high fraction does not bound unknown surface $U_A(I)$. Additional metrics should include newly discovered pathways per review pass, mutation survival, bypass success, false refusal, legitimate-variation loss, constraint-maintenance cost, and time to repair a discovered gap.

5.4 Counterfactual Baselines

At minimum, compare:

1. prose specification plus conventional tests;
2. prose plus expanded property, fuzz, mutation, and differential tests;
3. selected artifact bindings without receipts;
4. receipts without independent effect observation;
5. the full admissibility-engineering profile.

The comparison should hold task, consequence class, and evaluation corpus as constant as practical. It should measure defects found, defects surviving, false refusals, legitimate implementation diversity, human and machine cost, latency, and maintenance burden.

5.5 Negative Controls and Fault Injection

The evaluation should deliberately introduce missing interface methods, wrong postconditions, authority bypass, unreceipted mutation, forged or replayed receipt, stale configuration, hidden effect routes, suppressed specification amendment, incorrect reference behavior, and semantically wrong implementations that satisfy shape constraints. A technique earns credit only when its claimed layer detects the relevant injected defect without relying on undisclosed manual knowledge.

5.6 Evidence Gate

Until this package exists, the following remain hypotheses:

- plausible-but-wrong realization dominates mature agentic coding failures;
- a measurable complexity threshold separates prose-sufficient from artifact-bound regimes;
- the proposed bindings reduce residual defects more than expanded testing alone;
- receipt-backed evaluation improves causal confidence;
- cross-model, cross-path, or cross-language convergence predicts useful coding capability;
- the assurance gain justifies governance cost.

---

6. Evaluation and Convergence Profiles

6.1 Convergence Is Relation-Typed

Two realizations may converge under one relation and diverge under another. At least four relations must remain distinct:

1. **Exact artifact identity**: canonical artifact commitments are identical.
2. **Trace or receipt equivalence**: admitted traces are equivalent under a declared canonicalization and observation policy.
3. **Behavioral observational equivalence**: a declared observer cannot distinguish relevant behavior over the evaluation domain.
4. **Obligation satisfaction**: each realization independently satisfies the same invariant and evidence obligations.

Cross-language implementations ordinarily cannot share exact artifact hashes. They may satisfy the same obligations or produce traces related by a typed translation. Calling all four outcomes
`stream_hash convergence` would erase the property actually measured.

Represent a result as a vector rather than a binary label:

$$V = (a, t, b, o, d, c),$$

where a is artifact identity, t trace equivalence, b behavioral equivalence, o obligation satisfaction, d discovered saturation debt, and c measured cost. Each component names its decision procedure and evidence.

6.2 Evaluation Protocol

A comparative run should:

1. freeze the specification family and evaluation profile;
2. declare allowed configuration choices and tolerable ambiguity;
3. isolate hidden evaluation cases from executors where appropriate;
4. record all retries, human interventions, tool failures, and specification amendments;
5. execute negative, mutation, and bypass cases;
6. compute each convergence component independently;
7. report exclusions and missing observations;
8. reproduce a sample through an independent verifier.

Zero specification amendments is not automatically success. It may indicate a saturated specification, but it may also indicate suppressed feedback or an executor forced to implement a defect. Amendments must be typed as clarification, defect repair, scope change, configuration choice, or implementation accommodation.

6.3 What Becomes Comparable

Under a fixed evaluation profile, the method can compare models, prompts, toolchains, configuration choices, languages, and implementation paths. The comparison remains local to the tasks and consequence classes sampled. A result does not establish general model capability, universal language equivalence, or correctness outside the declared observer and threat model.

6.4 Polyglot Realization

A language-neutral invariant can be realized through language-specific binding guides. The useful question is not whether source artifacts are identical, but whether the guides preserve the same obligation set and whether the realized systems satisfy it under equivalent observations. Component-specific language selection is therefore an Instance Configuration decision subject to adapter, foreign-function, serialization, build, and operational boundary evidence.

6.5 Threats to Validity

Major threats include benchmark leakage, correlated executors, evaluator coupling, an incorrect reference model, underspecified equivalence, hidden human intervention, survivorship bias, changing toolchains, non-independent receipts, and an evaluation corpus derived from the same defects used to author the constraints. These should be reported as part of the result rather than treated as implementation notes.

---

7. Related Work and Positioning

This section locates admissibility engineering within the published research program and adjacent traditions. The relations are typed: AE extends, complements, or specializes prior work in defined ways.

Governed Agentic Coding (GAC) [Ableman 2026]. GAC establishes the *thrash threshold*–the phase transition at which directive authority collapses within an agentic coding loop under context decay. AE introduces a specification-side complement: the *complexity threshold* at which prose specifications cease to close the saturation gap. GAC governs loop length; AE governs specification saturation. The two thresholds are independent failure modes; both must be addressed for stable agentic development.

Receipt-Bound Capabilities (RBC) [Ableman 2026]. RBC defines the discipline of inseparably binding permission to act and obligation to produce evidence. Section 4 of this paper builds directly on RBC: the receipt chain as governance substrate is the architectural realization of RBC at the system scale.

Governance Coverage Gaps (GCG) [Ableman, forthcoming]. GCG identifies failures in *governance scope*: a system's failure to declare governance

over capabilities it exercises. AE identifies failures in *constraint surface*: a specification's failure to constrain the realization surface of its declared invariants. The two are complementary diagnostics: GCG asks whether the right things are governed; AE asks whether what is governed is governed completely.

The Constitutional Codex [Ableman, forthcoming]. The Codex paper establishes the document type that makes governed AI-assisted construction possible. AE describes the discipline by which Codices reach ideal saturation above the complexity threshold–the *how* of writing a Codex that converges on a system whose complexity exceeds prose-only governance.

Constraint-Bound Inference (CBI) [Ableman 2026]. CBI characterizes the inferential mode of executors operating within declared constraints. AE constructs the constraint surfaces on which CBI operates above the complexity threshold; CBI is the inferential mechanism, AE is the constraint discipline.

Testing for Deriving Observable State [Ableman, forthcoming]. This paper grounds the evidence-vs-behavior distinction that Section 4 of AE depends on: tests must verify evidence, not just behavioral outputs. The receipt-chain-as-substrate architecture is the operational realization.

Constitutional Agentic Compilation (CAC) [Ableman, forthcoming]. CAC introduces bounded-discretion realization from constitutional specifications. AE is the discipline that makes bounded discretion operational above the complexity threshold; without AE, "bounded discretion" remains aspirational because the specification cannot bound the discretion tightly enough.

Formal methods (TLA+, Alloy, Z) [Lamport 2002; Jackson 2012]. Formal methods aspire to language-agnostic correctness through mathematical specification. They provide rigorous languages and analyses but require specialized modeling and still leave refinement, implementation, environment, and assurance-boundary obligations. AE is more permissive–natural language plus artifact and evidence constraints–and should be evaluated as a complementary engineering discipline, not as a demonstrated substitute for formal proof. Their useful composition and division of labor are empirical and domain-specific questions. The relation is one of operational complementarity: formal methods for the highest-assurance components, AE for the system as a whole.

Design by Contract [Meyer 1992]. DbC introduces preconditions, postconditions, and invariants at the method level. Implementation contracts in Section 3.3 are direct descendants of DbC. AE generalizes the DbC discipline to the system level: phases have preconditions (prerequisites), postconditions (exit criteria), and invariants spanning multiple phases.

LLM-assisted code generation. Agentic and non-agentic coding systems use heterogeneous generation, search, test, repair, and review loops. AE proposes an evaluation and constraint discipline that can be applied across those loops. A current external literature review is still required for a future evidence-qualified revision to compare it fairly with contemporary repository-level agents,

specification-guided synthesis, proof-carrying code, refinement types, typestate, property-based testing, mutation testing, and formal verification.

7.1 Ableman Corpus and Realization-Funnel Placement

The current meaning architecture is the **Governed Meaning Base (GMB)** under the author-confirmed historical lineage `CCF -> CMB -> GMB`. GMB can hold the identity-bearing invariants, relations, standing, evidence links, and revision history that AE projects into a bounded construction task. It does not make those projections correct merely by storing them.

The adjacent corpus roles are:

Work or structure	Relation to admissibility engineering
Ableman's Razor / Realization Funnel	Separates possibility, specification, configuration, realized implementation, evidence, and revision so that a constraint cannot silently migrate between stages.
Constraint-Bound Inference	Describes execution within bounds; AE constructs and tests the bounds.
Meaning as Binding	Distinguishes semantic availability from material that has valid standing to bind realization.
Constitutional Context Compilation	Governs which specifications, evidence, tools, and prior decisions enter an agent invocation.
The Three Hash Rule	Prevents object state, admissibility, and meaning commitments from collapsing into one apparently sufficient digest.
Reasoning by Way of Receipts	Preserves intermediate decisions, alternatives, and debts during construction.
Semantic and Work Custody	Track who may transform meaning and who holds the bounded duty to advance the exact work head.
Governed Semantic Project Lineage	Binds project heads, rebuildability, cold-boot capsules, and successor relations across sessions.

AE traverses the funnel as follows:

Stage	AE responsibility
Possibility	Enumerate competing realizations, including harmful, brittle, and overconstrained ones.
Specification	Declare invariants, pathways, tolerable ambiguity, obligations, and refusal semantics.
CM1	Enumerate configurations satisfying the bare specification, including undesirable admissible cases.
CM2	Prefer configurations using safety, sovereignty, maintainability, portability, cost, and legitimate-freedom criteria.
Instance Configuration	Select actual bindings, languages, tools, thresholds, test profiles, adapters, and observers.
Implementation Realization	Construct the system and expose actual capability and effect pathways.
Evidence	Test constraints, effects, bypasses, equivalence relations, cost, and residual unknowns.
Revision	Feed counterexamples and operational evidence back into inventories, constraints, configurations, and claims.

This paper primarily defines the Specification discipline and the transition into CM1/CM2. It supplies an evidence protocol, not a certified implementation realization.

7.2 Counterevidence and Compression Test

AE's distinctiveness is reduced if expanded property, mutation, differential, and formal testing closes the same defect classes without its additional vocabulary; if pathway enumeration proves too unstable to support meaningful denominators; if artifact bindings primarily shift defects into the binding artifacts; if legitimate implementation diversity collapses; if receipt requirements add cost without increasing effect confidence; or if convergence scores fail to predict maintenance, security, or user value.

A strong evaluation should therefore include simpler baselines and report mechanisms removed after they fail to contribute independent assurance. Saturation of the paper's own architecture requires authorial compression as much as implementation constraint.

Pathway-Constraint Saturation Discipline

Across the completed P1 corpus, Admissibility Engineering is the **engineering discipline for discovering consequence-bearing realization pathways, binding the least restrictive adequate constraints at appropriate layers, independently challenging those bindings, and revising the observed saturation gap**. It does not prove absolute completeness, replace testing with artifacts, or make receipts equivalent to behavior.

- **Adjacent work:** *The Implementation Codex*
  Supplies: Selected dependencies, phases, bindings, gates, evidence, and journal.
  AE boundary: AE evaluates whether the package constrains the relevant realization surface.
- **Adjacent work:** *Governance Coverage Gaps*
  Supplies: Capabilities and effects omitted from the declared governance membrane.
  AE boundary: GCG asks what escaped scope; AE asks what remains unconstrained inside declared scope.
- **Adjacent work:** *Agentic Interface Surfaces*
  Supplies: Action, effect, evidence, feedback, and recognition surfaces.
  AE boundary: AIS exposes agent-facing enforcement; AE tests its pathway coverage and false decisions.
- **Adjacent work:** *The Mechanics of Constitutional Enforcement*
  Supplies: Rule placement, mediation, courts, effect atomicity, and classified remainder.
  AE boundary: AE supplies the saturation method for those enforcement locations.
- **Adjacent work:** *Reasoning by Way of Receipts*
  Supplies: Durable transition and evidence records with claim ceilings.
  AE boundary: AE uses receipts in plural evidence without trusting them as complete effects.

An operational saturation claim should bind:

$$\begin{aligned} S = (&architecture, invariant, consequence_class,\\ &pathway_inventory, enumeration_method,\\ &constraint_bindings, evidence, ambiguity, bypasses,\\ &unknown_surface, counterexamples, cost, revision) \end{aligned}.$$

The observed gap is only over enumerated pathways. New capabilities, adapters, retries, error paths, providers, migrations, or physical effects can reopen the surface. Tolerable ambiguity must be declared so legitimate implementation diversity is not mistaken for a defect or silently destroyed by overconstraint.

Every constraint relocation has authority and migration consequences. Types, schemas, contracts, mediators, oracles, receipts, tests, and models can each encode the wrong invariant or share one failure mode. The full profile therefore compares against expanded testing and simpler baselines, measures false admission and false refusal, and removes machinery that contributes no independent assurance.

The universal GSM can type
`ENUMERATES_REALIZATION_PATHWAY`, `BINDS_INVARIANT_AT_LAYER`, `COVERS_PATHWAY_WITH_EVIDENCE`, `LEAVES_OBSERVED_SATURATION_GAP`, `PRESERVES_TOLERABLE_AMBIGUITY`, `DISCOVERS_UNKNOWN_PATHWAY`, `BYPASSES_CONSTRAINT_BINDING`, `FALSELY_REFUSES_REALIZATION`, `REOPENS_SATURATION_CLAIM`, and `COMPRESSES_REDUNDANT_MECHANISM`. These relations make saturation defeasible and measurable.

This integration establishes four editorial commitments: classify AE as the pathway-constraint saturation discipline; adopt the operational-saturation envelope; require explicit tolerable ambiguity, unknown surface, simpler baselines, and false-decision costs; and make every saturation claim reopenable by new pathways, counterexamples, or architecture revisions.

Capability-Specific Admissibility and Cross-Domain Term Firewall

The fixed-seed comparison with the Cognitive ABI, Crystallized Governance, the international-trade UGK profile, and Admissibility Theory of Consciousness makes admissibility explicitly typed:

$$\begin{aligned} A = (&subject, domain, jurisdiction, requirement,\\ &capability_contract, architecture, pathways,\\ &constraint_bindings, evidence, false_admission,\\ &false_refusal, unknowns, revision) \end{aligned}.$$

The Cognitive ABI names the capability contract and eligible realization conditions. Crystallized Governance supplies source-correspondence, remainder, adoption, contest, and amendment obligations that may constrain a realized pathway. Neither layer proves pathway coverage; AE inventories adapters, retries, fallbacks, caches, effectors, and error paths for the selected architecture.

The international-trade material offers a bounded application surface for mandate, documentary claim, handoff, amendment, refusal, and jurisdictional decision. A trade pilot would need domain authority and legal validation in addition to engineering saturation. The current candidate title remains an AI proposal over an untitled source head and does not acquire author standing here.

Admissibility Theory of Consciousness uses admissibility as a structural research vocabulary about possible, viable, reflexive, and self-authoring organization. AE uses it as an engineering decision over a named realization and requirement. Shared vocabulary does not transfer evidence, thresholds, mechanisms, or phenomenality claims between the two.

The universal GSM can type
`EVALUATES_TYPED_ADMISSIBILITY`, `BINDS_CAPABILITY_PATHWAY`, `IMPLEMENTS_CONSTITUTIONAL_OBLIGATION`, `PROFILES_TRADE_REALIZATION`, `DISTINGUISHES_ADMISSIBILITY_DOMAIN`, `REFUSES_CROSS_DOMAIN_EVIDENCE_TRANSFER`, and
`REOPENS_ON_ARCHITECTURE_CHANGE`.

This randomized review establishes four editorial commitments: adopt the domain-typed AE object; bind saturation to capability contracts and constitutional obligations; retain trade as a separately authorized application profile; and install a terminology firewall between engineering admissibility and consciousness research.

A Reopenable Architecture-Relative Saturation Ledger

Twenty retained downstream uses place Admissibility Engineering across capability contracts, quant systems, collective coordination, context management, governance crystals, frozen compilers, physical control, possibility analysis, epistemic profiles, ACG courts, coverage gaps, incentive design, GCNL, anti-self-ratification, MosaiCOS, semantic binding quality, semantic debt, split authority, AFS, and COP. They confirm a cross-cutting engineering discipline for pathway discovery and constraint placement, not a claim of absolute completeness.

$$\begin{aligned} A^* = (&subject, generation, requirement, consequence_class,\\ &architecture, capability_surface, effect_surface,\\ &pathway_inventory, enumeration_method,\\ &constraint_bindings, placement_authority, evidence,.\\ &bypasses, false_admission, false_refusal,\\ &tolerable_ambiguity, unknown_surface, cost,\\ &reopen_triggers, revision) \end{aligned}$$

Saturation is always relative to an exact architecture, generation, requirement, threat model, and enumerated pathway set. New adapters, providers, retries, caches, error paths, migration steps, privileges, or physical effects reopen the claim. A constraint moved into a type, schema, model, compiler, mediator, test, receipt, or oracle changes both failure mode and authority topology.

The discipline must optimize a two-sided and noncompensatory profile: harmful false admission, legitimate false refusal, unobserved pathway, residual ambiguity, performance and maintenance cost, privacy, portability, sovereignty, and

repairability. More constraints are not automatically better. Independent challenge must include mutation, bypass, shared-oracle, stale-policy, relocation, and simpler-baseline tests.

The universal GSM can type
`DECLARES_SATURATION_SUBJECT`,
`ENUMERATES_REALIZATION_PATHWAY`, `BINDS_CONSTRAINT_AT_LAYER`,
`ASSIGNS_PLACEMENT_AUTHORITY`, `DISCOVERS_BYPASS`, `MEASURES_FALSE_ADMISSION`,
`MEASURES_FALSE_REFUSAL`, `PRESERVES_TOLERABLE_AMBIGUITY`, `REOPENS_ON_ARCHITECTURE_CHANGE`, and
`COMPRESSES_REDUNDANT_ASSURANCE`.

This review establishes four editorial commitments: adopt the complete reopenable saturation ledger; bind each claim to architecture, generation, requirement, consequence, threat, and pathway set; evaluate false admission, false refusal, ambiguity, unknown surface, and cost separately; and require authority-aware constraint placement, independent attacks, and simpler baselines.

Constraints Can Be Wrong Too

Public orientation

Admissibility Engineering asks how an AI coding process can make prohibited outcomes structurally unavailable rather than merely discouraged in prose. Its central move is valuable only if the admitted region is itself reviewable: a machine can enforce the wrong boundary with perfect consistency.

Operative definition

For a declared profile p, request x, realization r, obligations O_p, and tests T_p, define:

$$A_p(r, x) = 1 \quad \text{only if} \quad r \models O_p \ \wedge \ T_p(r, x) = \text{PASS}.$$

`PASS` means admitted under profile p; it does not mean optimal, safe in every environment, semantically correct beyond the tested obligations, or authorized for deployment. `FAIL`,
`UNKNOWN`, `CONFLICT`, `OUT_OF_SCOPE`, and `UNTESTED_REMAINDER` are distinct outcomes.

Adversarial tests

1. **Wrong-invariant attack.** A faithfully enforced invariant encodes a mistaken requirement. The profile must carry source, authority, revision, challenge, and supersession rather than treating enforcement as correctness.
2. **Overconstraint attack.** The profile excludes legitimate realizations and collapses useful diversity. Rejected regions require inspectable reasons and a governed amendment route.

3. **Shared-failure convergence.** Independent realizations produce the same digest or behavior because the specification, oracle, fixture, or environment is wrong. Convergence is evidence about relation to a fixed test surface, not truth.
4. **Independence theater.** Realizations described as independent share prompts, libraries, evaluators, fixtures, or model lineage. Independence must be dimension-typed and evidenced.
5. **Metric gaming.** A realization optimizes the declared evaluator while violating unstated purpose. The result exposes specification debt; it does not justify silently widening the oracle.
6. **Polyglot drift.** Different runtimes satisfy surface tests while diverging at serialization, arithmetic, exception, scheduling, or dependency boundaries. The equivalence profile must name these semantics.

Claim and evidence ceiling

The paper establishes a method and vocabulary for relocating obligations into inspectable artifacts. It does not yet establish a general reduction in defects, cost, energy, review burden, or security incidents. Stream-hash equality is a strong result only for subjects whose canonicalization and environmental closure are demonstrated. Behavioral agreement is bounded by the observation surface.

Internal examples and corpus connections explain how the method composes with constitutional computing. Publication claims about comparative performance, reliability, or organizational outcomes require external or newly produced evidence. No external research was added in this pass.

Reader-facing consequence

The public value is not "more rules for AI." It is the ability to state what a realization must preserve, test those obligations across implementations, refuse malformed results, and reopen the rules when the rules themselves fail. That distinction should lead the paper's abstract and conclusion.

8. Conclusion

Admissibility engineering addresses a real and often hidden problem: a system may satisfy the checks that exist while violating an obligation the specification failed to bind to the realized pathway.

The discipline proposed here is not "replace prose with code" and not "trust receipts instead of tests." It is a governed loop of pathway discovery, constraint placement, independent testing, effect observation, evidence binding, counterexample search, and revision. A saturation claim is always relative to an exact architecture, configuration, capability boundary, threat model, and evidence package.

This reframing preserves the paper's central insight while limiting its claim. Artifact-bound constraints can reduce some forms of plausible-but-wrong realization. They can also encode mistakes, hide authority, constrain legitimate variation, or be bypassed. Receipts can make transitions reviewable; they cannot prove their own completeness or the truth of the effects they report. Convergence can support comparison only after the relevant equivalence relation is named.

The immediate research task is empirical: bind the current systems and codices to exact heads, construct pathway and constraint matrices, execute fault-injection and simpler-baseline comparisons, and measure defect survival, false refusal, legitimate variation, governance cost, and maintenance burden. Only then can the dominance, threshold, convergence, and comparative-capability hypotheses receive evidentiary standing.

The long-term aim remains powerful: specifications that preserve intentional freedom while making consequence-bearing wrong realizations detectably inadmissible. Ambiguity engineering is the complementary discipline–not the absence of saturation, but the deliberate preservation of choice within a proven boundary.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Cm2.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.
- **Instance Configuration.** The present paper does not claim a complete instance configuration treatment. That stage remains successor work and cannot be inferred from the architecture described here.
- **Revision.** The present paper does not claim a complete revision treatment. That stage remains successor work and cannot be inferred from the architecture described here.

Corpus-Wide Reference Positioning

This paper argues that agentic software construction can produce implementations that compile, run, and pass the available tests while violating an invariant that the specification intended to preserve. Its nearest external comparison fields are formal, provenance, software. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical

method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. A complete provenance graph can faithfully record a false or unauthorized process; provenance is evidence about lineage, not automatic truth or legitimacy. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E2**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

[**E3**] Patrick Cousot and Radhia Cousot. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." *POPL '77*, 1977. DOI: 10.1145/512950.512973. https://doi.org/10.1145/512950.512973 **Relationship:** Predecessor formal framework for sound approximation and fixpoint reasoning.

[**E4**] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. https://lamport.azurewebsites.net/tla/book.html **Relationship:** Predecessor specification discipline for state machines, safety, liveness, and refinement.

[**E5**] W3C. *PROV-O: The PROV Ontology*. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-o/ **Relationship:** Adjacent provenance standard; it models entities, activities, and agents, but does not by itself establish the paper's stronger authority or standing claims.

[**E6**] Santiago Torres-Arias et al. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *28th USENIX Security Symposium*, 2019, pp. 1393-1410. usenix.org source **Relationship:** Predecessor supply-chain lineage system used for comparison; no influence claim is made unless the source head already documents one.

[**E7**] OpenSSF. *Supply-chain Levels for Software Artifacts (SLSA), Version 1.2*. 2026. https://slsa.dev/spec/v1.2/ **Relationship:** Adjacent software-supply-chain assurance standard, especially relevant to provenance and verification boundaries.

Ableman corpus

[**A1**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation*. Ableman Corpus current edition; work identity `PAPER.PUB.21095303`;
source SHA-256
`EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Semantic Compilation*. Ableman Corpus current edition; work identity `PAPER.PUB.18453681`;
source SHA-256
`5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Hybrid Semantic Compiler Architectures*. Ableman Corpus current edition; work identity `PAPER.PUB.21228038`;
source SHA-256
`EBCD6DDDA9B547639AEBED79FB68069473CF1248E41C527B48C9624BBE2C0496`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Everything Is a Compiler.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-30FB8F95BD990A47`;
source SHA-256
`FE0F88A1F125317285BE4378E5F27ACC9FEB83ADE3FF329A3DA9F9CB026DF9A9`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Reasoning by Way of Receipts.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-04E427F6F4D94454`;
source SHA-256
`AF86A66CCD6A103DD75208500AE99148911A6453B6DFDF36A954279F21AB8489`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *The Epistemic Lineage Engine.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-E0EC3DC7EABF5060`;
source SHA-256
`CDD74D9AFAB6532645E68F7F6C9F0F7A9E472D9B8E717290522566FF8DC01F11`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *The Three Hash Rule.* Ableman Corpus current edition; work identity `PAPER.PUB.20788617`;
source SHA-256
`F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Adam Ableman *Ableman's Razor: A Typed Grammar for Ontology Exploration, Configuration Collapse, and Implementation Feedback.* Zenodo, 2026. doi:10.5281/zenodo.18113475

Adam Ableman *Constraint-Bound Inference and Invariant Extraction.* Zenodo, 2026. doi:10.5281/zenodo.18175270

Adam Ableman *Governed Agentic Coding (GAC): Thrash Threshold, Churn, and In-Loop Epistemic Authority Governance for Frontier AI Coding Systems.* Zenodo, 2026. doi:10.5281/zenodo.18629665

Adam Ableman *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality.* Zenodo, 2026. doi:10.5281/zenodo.18601631

Adam Ableman *The Constitutional Codex: AI-Executable Specification and Phase-Gated Construction for Deterministic System Realization.* Forthcoming.

Adam Ableman *Governance Coverage Gaps: Diagnosing the Absence of Governance at Capability Boundaries.* Forthcoming.

Adam Ableman *Testing for Deriving Observable State of Software Systems.* Forthcoming.

Adam Ableman *Constitutional Agentic Compilation: Bounded-Discretion Realization from Constitutional Specifications.* Forthcoming.

Lamport, L. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley, 2002.

Jackson, D. *Software Abstractions: Logic, Language, and Analysis.* MIT Press, 2012.

Meyer, B. "Applying 'Design by Contract.' " *IEEE Computer* 25(10), pp. 40–51, 1992.

Adam Ableman *Governed Context & Memory (GCM): Context Configuration as Constitutional Surface, Semantic Hashing as Enforcement, and Explicit Ignorance as First-Class Success in Generational AI.* Zenodo, 2026. doi:10.5281/zenodo.18633797

Adam Ableman *Semantic Custody.* Zenodo, 2026. doi:10.5281/zenodo.20300952

Adam Ableman *The Three Hash Rule.* Zenodo, 2026. doi:10.5281/zenodo.20788617

Adam Ableman *Governed Meaning Base (GMB).* Ableman Corpus current edition in the historical CCF -> CMB -> GMB lineage; conceptual claims retain the standing stated in the work.

Adam Ableman *Meaning as Binding.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

Adam Ableman *Reasoning by Way of Receipts.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

Adam Ableman *From Content-Addressed Storage to Governed Semantic Project Lineage.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23847370